



Git & GitHub Interview Questions

✦ Asked-in-Interviews + Real-World Implementation Notes ✦



@2005_vaibhavsingh



52
Questions



52
Pro Tips



52
Real Uses



Foundations (Q1–Q13)



? Q1. Can you explain what Git is and why teams use it?

Answer:

Git is a distributed version control system (DVCS) that helps track code changes, collaborate with teams, roll back to older versions, and manage branching/merging.



Pro Tip: Always mention 'distributed' — it highlights Git's offline capability.

 **Real-World Use:** Every modern dev team uses Git daily to track every code change, revert bad deploys, and let multiple developers work on the same codebase without overwriting each other.

? Q2. What is GitHub and how is it different from Git itself?

 **Answer:**

GitHub is a cloud platform to host Git repositories and collaborate using Pull Requests, Issues, Actions (CI/CD), and project boards.

 **Pro Tip:** Say Git = tool, GitHub = platform.

 **Real-World Use:** Teams host their company codebase on GitHub, use PRs for code review before anything reaches production, and use Issues to track bugs reported by QA or users.

? Q3. Walk me through the Git commands you use in your daily workflow.

 **Answer:**

 **Commands / Steps:**

- ▶ git init → Initialize a new Git repository
- ▶ git clone <url> → Copy repository from remote to local
- ▶ git status → Show modified/untracked files
- ▶ git add . → Stage all changes
- ▶ git commit -m 'msg' → Save a snapshot with a message
- ▶ git log --oneline → Show commit history in short form
- ▶ git branch → List branches
- ▶ git checkout -b <branch> → Create + switch to new branch
- ▶ git merge <branch> → Merge branch into current
- ▶ git push origin main → Push local commits to GitHub
- ▶ git pull → Fetch + merge changes from remote
- ▶ git fetch → Only download changes (no merge)

 **Pro Tip:** Mention git commit -am 'msg' as a shortcut for tracked files.

 **Real-World Use:** In a typical workday you'll clone the repo once, then repeat add → commit → push for every feature, and pull before starting new work.

? Q4. In your own words, how would you differentiate Git from GitHub?

 **Answer:**

Git is a version control system that runs locally on your machine and helps you track changes. GitHub is a cloud platform that uses Git, and provides remote repositories, collaboration tools, pull requests, issues, and more.

 **Pro Tip:** Think: Git = tool, GitHub = platform.

 **Real-World Use:** You can use Git fully offline, but you need GitHub the moment you want to collaborate with a remote team.

? Q5. Why do developers create separate branches instead of working on main directly?

 **Answer:**

A branch is a lightweight movable pointer to a commit. It allows developers to work on features or bug fixes independently without affecting the main codebase.

 **Pro Tip:** Use meaningful branch names like 'feature/login-page' or 'fix/payment-bug'.

 **Real-World Use:** At most companies, main/production is protected — every developer creates a feature branch so a bug in their code can't break the live app.

? Q6. How do you create a new branch and switch to it?

 **Answer:**

 **Commands / Steps:**

- ▶ `git branch <branch-name>` → Create a new branch
- ▶ `git checkout <branch-name>` → Switch to branch
- ▶ `git checkout -b <branch-name>` → Create & switch in one step
- ▶ `git switch -c <branch-name>` → Modern alternative (Git 2.23+)

 **Pro Tip:** Newer Git versions also support `git switch -c <branch-name>`.

 **Real-World Use:** Before picking up a Jira ticket, you'd run `git checkout -b feature/JIRA-204-login-fix` so your work is isolated and easy to review.

? Q7. What happens when you merge two branches, and when would you do it?

 **Answer:**

Merging combines changes from one branch into another. You typically checkout the target branch first, then run `git merge`.

 **Commands / Steps:**

- ▶ `git checkout main`
- ▶ `git merge <branch-name>`

 **Pro Tip:** Always pull the latest changes before merging to avoid conflicts.

 **Real-World Use:** Once a feature is reviewed and approved in a PR, it gets merged into `develop` or `main` as part of the release process.

? Q8. Have you faced a merge conflict? How did you resolve it?

 **Answer:**

A merge conflict occurs when Git cannot automatically resolve differences in code between two branches. You must manually resolve it.

 **Commands / Steps:**

- ▶ Manually edit the conflicting files

- ▶ Remove conflict markers (<<<<<<, =====, >>>>>>)
- ▶ git add <file>
- ▶ git commit

 **Pro Tip:** In interviews, mention git commit -am 'msg' as a shortcut for tracked files.

 **Real-World Use:** This happens constantly when two developers edit the same lines — interviewers want the exact step-by-step process you followed.

? Q9. What is a pull request and why is it central to team workflows?

 **Answer:**

A pull request is a way to propose changes from one branch to another and request that those changes be reviewed and merged.

 **Pro Tip:** Write clear PR descriptions and link related issues for better reviews.

 **Real-World Use:** No code reaches production without a PR — it's where senior engineers review your code, leave comments, and approve before merging.

? Q10. When have you used git stash in real projects?

 **Answer:**

Stash temporarily saves your uncommitted changes so you can work on something else and reapply them later.

 **Pro Tip:** Use git stash list to see stashes and git stash pop to reapply and remove.

 **Real-World Use:** You're mid-feature when an urgent production bug arrives — stash your half-done work, switch branches to fix the bug, then pop your stash to resume.

? Q11. How do reset, revert, and restore differ — and which undo a bad pushed commit?

💬 Answer:

⚙️ Commands / Steps:

- ▶ git reset → Moves HEAD to another commit. Can rewrite history. LOCAL only.
- ▶ git revert → Creates a new commit that undoes changes. SAFE for shared branches.
- ▶ git restore → Discards changes in working directory or staging area.

💡 **Pro Tip:** Use revert for public/shared history, reset for local cleanup only.

🌐 **Real-World Use:** On a shared branch you'd always use revert — using reset on pushed history can break your teammates' local repos.

? Q12. What's the role of forking in open-source contribution?

💬 Answer:

Forking creates a copy of a repository under your GitHub account. It's commonly used to contribute to open source projects.

💡 **Pro Tip:** Keep your fork up to date with the original repository using an upstream remote.

🌐 **Real-World Use:** To contribute to a public repo you don't have write access to, you fork it, make changes, then open a PR back to the original project.

? Q13. Why is .gitignore important in a real project?

💬 Answer:

.gitignore specifies intentionally untracked files that Git should ignore — such as build artifacts, secrets, and IDE configs.

💡 **Pro Tip:** Add build files, logs, .env files, and IDE settings to .gitignore.

 **Real-World Use:** Without it you'd accidentally commit `node_modules`, `.env` files with secrets, or build artifacts — a common rookie mistake interviewers ask about.

Branching & Merging (Q14–Q25)

? Q14. When and why would you create a Git tag?

 **Answer:**

A tag marks a specific point in history as important, usually for release versions. Tags come in lightweight and annotated forms.

 **Pro Tip:** Use annotated tags (`git tag -a v1.0 -m 'Release'`) with messages for better clarity.

 **Real-World Use:** Release engineers tag every production release like `v2.3.0` so they can instantly check out exactly what code was shipped on any date.

? Q15. What's a Git remote, and how is it used in a typical setup?

 **Answer:**

A remote is a version of your repository hosted on the internet or network. 'origin' is the default remote name when you clone a repo.

 **Pro Tip:** Use meaningful remote names like 'origin' for your fork, 'upstream' for the source.

 **Real-World Use:** Your local repo usually has one remote called `origin` pointing to your company's GitHub/GitLab server.

? Q16. git pull vs git fetch — when would you deliberately choose fetch?

💬 Answer:

⚙️ Commands / Steps:

- ▶ git fetch → Downloads changes from remote but does NOT merge.
- ▶ git pull → Fetches changes AND merges into your current branch.

💡 **Pro Tip:** Use fetch to review incoming changes with git diff before merging.

🌐 **Real-World Use:** Senior developers often fetch first to review what's coming in before merging, avoiding surprise conflicts.

? Q17. Describe your exact process for resolving a merge conflict.

💬 Answer:

Step-by-step process:

⚙️ Commands / Steps:

- ▶ Git marks conflicting files automatically.
- ▶ Open the file and find conflict markers (<<<<<<, =====, >>>>>>).
- ▶ Edit the file to keep the correct/combined changes.
- ▶ Save the file and run: git add <file>
- ▶ Commit the merge: git commit

💡 **Pro Tip:** Communicate with your team to prevent conflicts before they happen.

🌐 **Real-World Use:** This is one of the most commonly whiteboarded practical questions — interviewers want the step-by-step, not just theory.

? Q18. How have you used GitHub Actions in a project?

💬 Answer:

GitHub Actions is a CI/CD tool that automates workflows like testing, building, and deploying your code. Workflows are defined in YAML files.

 **Pro Tip:** Store workflow files in the `.github/workflows` directory.

 **Real-World Use:** A typical pipeline runs tests and linting on every PR, then auto-deploys to staging when code merges to main.

? Q19. When would you choose rebase over merge in a real team workflow?

 **Answer:**

Rebase moves or replays a sequence of commits onto a new base, creating a linear project history instead of a merge commit.

 **Pro Tip:** Never rebase commits that have already been pushed to a shared branch.

 **Real-World Use:** Before opening a PR, developers often rebase their feature branch onto the latest main to keep history clean.

? Q20. Merge vs Rebase — what's the tradeoff, and which does your team prefer?

 **Answer:**

 **Commands / Steps:**

- ▶ Merge → Combines histories, creates a merge commit. Preserves full history.
- ▶ Rebase → Replays commits on top of another branch. Produces clean linear history.

 **Pro Tip:** Use merge for shared branches, rebase for cleaning up local feature branches before a PR.

 **Real-World Use:** Teams that value a clean linear history enforce rebase-before-merge as a PR rule across the org.

? Q21. Give an example of when you'd use cherry-pick on the job.

 **Answer:**

Cherry-pick applies a specific commit from one branch onto another without merging the entire branch.

 **Pro Tip:** Useful for backporting a bug fix to a release branch.

 **Real-World Use:** A critical bug fix lands on develop, but you need it live on the production hotfix branch immediately — cherry-pick brings just that one commit over.

? Q22. What does HEAD represent, and why does it matter when switching branches?

 **Answer:**

HEAD is a pointer that refers to the current commit or branch you are working on in your repository.

 **Pro Tip:** HEAD usually points to the latest commit of the currently checked-out branch.

 **Real-World Use:** Confusing HEAD's position is the #1 cause of 'I lost my commit!' panic among junior developers — understanding it prevents that.

? Q23. Have you ever ended up in a detached HEAD state? What did you do?

 **Answer:**

A detached HEAD occurs when you check out a specific commit instead of a branch, so HEAD points directly to a commit rather than a branch reference.

 **Pro Tip:** Create a new branch before committing in a detached HEAD state, or your commits may be lost.

 **Real-World Use:** This typically happens when checking out an old commit to debug a regression — you'd create a branch immediately to save any new work.

? Q24. How do you review your changes before committing?

 **Answer:**

 **Commands / Steps:**

- ▶ `git diff` → Shows unstaged changes vs working directory
- ▶ `git diff --staged` → Shows staged changes vs last commit
- ▶ `git status` → Overview of what's staged, unstaged, and untracked

 **Pro Tip:** Use `git diff --staged` right before every commit to catch accidental debug code.

 **Real-World Use:** Running `git diff --staged` before every commit is a habit most senior engineers use to avoid committing leftover `console.logs`.

? Q25. How would you find out who introduced a specific bug in the code?

 **Answer:**

`git blame` shows who last modified each line of a file and in which commit, making it easy to trace the origin of a bug.

 **Pro Tip:** Combine with `git log -p` for more context around a suspicious change.

 **Real-World Use:** QA reports a regression — `git blame` on the broken line instantly tells you which commit and developer to talk to.

⚡ Advanced Git (Q26–Q39) ⚡

? Q26. How do you inspect exactly what changed in a specific commit?

💬 Answer:

git show displays detailed information about a specific commit, including the diff it introduced, the author, and timestamp.

💡 **Pro Tip:** Run git show <tag> to see what an annotated tag points to.

🌐 **Real-World Use:** Useful during code audits or when reviewing a teammate's commit referenced in a PR discussion.

? Q27. What's actually stored inside the hidden .git folder?

💬 Answer:

The .git directory stores all metadata and the object database for the repository.

⚙️ **Commands / Steps:**

- ▶ HEAD → Current branch pointer
- ▶ objects/ → All commits, trees, and blobs
- ▶ refs/ → Branch and tag references
- ▶ config → Repository-specific settings

💡 **Pro Tip:** Never manually edit files inside .git unless you know exactly what you're doing.

🌐 **Real-World Use:** Understanding this helps when a repo gets corrupted or oversized — experienced engineers know what's safe to clean up.

? Q28. How do you explore a project's commit history effectively?

💬 Answer:

⚙️ Commands / Steps:

- ▶ `git log --oneline` → Compact single-line view
- ▶ `git log --graph --all` → Visual branch graph
- ▶ `git log -p` → Show diffs per commit
- ▶ `git log --author='name'` → Filter by author
- ▶ `git log --since='2 weeks ago'` → Filter by date

💡 **Pro Tip:** Use `git log --oneline --graph --decorate --all` for a quick full visual history.

🌐 **Real-World Use:** Before starting work on legacy code, developers run a graph log to understand the branching and release pattern of the project.

? Q29. When would you amend a commit instead of creating a new one?

💬 Answer:

`git commit --amend` lets you modify the most recent commit — changing the message or adding forgotten changes, without creating a new commit.

💡 **Pro Tip:** Never amend a commit that has already been pushed and shared with others.

🌐 **Real-World Use:** You just committed and immediately spotted a typo in the message or forgot to add a file — `amend` fixes it cleanly.

? Q30. Why would a CI/CD pipeline use a shallow clone?

💬 Answer:

A shallow clone downloads only a limited number of recent commits instead of full project history, making the clone faster and smaller.

💡 **Pro Tip:** Use `git clone --depth 1 <url>` for quick CI builds that don't need full history.

 **Real-World Use:** Cloning a huge multi-year repo on every build wastes minutes — CI servers use `--depth 1` to speed up pipeline runs significantly.

? Q31. When have submodules been useful (or painful) in your experience?

 **Answer:**

Submodules let you include and track another Git repository as a subdirectory inside your main repository, pinned to a specific commit.

 **Pro Tip:** Run `git submodule update --init --recursive` after cloning a repo with submodules.

 **Real-World Use:** Companies use submodules to share a common internal library across multiple repos, though many now prefer monorepos to avoid the complexity.

? Q32. How would you track down which commit introduced a regression in a large codebase?

 **Answer:**

`git bisect` performs a binary search through commit history to find the commit that introduced a bug in just a few steps.

 **Pro Tip:** Mark commits with `git bisect good` and `git bisect bad` to narrow it down automatically.

 **Real-World Use:** Instead of manually checking 50 commits, `git bisect` narrows it down in about 6 binary-search steps.

? Q33. Why might a team enforce `pull.rebase=true` company-wide?

 **Answer:**

 **Commands / Steps:**

- ▶ `git pull` → Fetches and merges, creating a merge commit if histories diverge.

- ▶ `git pull --rebase` → Fetches and replays your local commits on top of remote branch.

 **Pro Tip:** Set `pull.rebase=true` in config if your team prefers linear history by default.

 **Real-World Use:** It keeps the commit graph linear across the whole org, which makes git log and code archaeology much easier.

? Q34. Where have you used `.gitattributes` in a cross-platform team?

 **Answer:**

`.gitattributes` defines attributes for file paths: line-ending normalization, diff behaviour, and marking files as binary.

 **Pro Tip:** Use it to enforce consistent line endings (LF) across different operating systems.

 **Real-World Use:** Windows and Mac developers on the same team get CRLF/LF line-ending conflicts constantly — `.gitattributes` standardizes it.

? Q35. Have you set up Git hooks before? What for?

 **Answer:**

Git hooks are scripts that run automatically at specific points in the Git workflow.

 **Commands / Steps:**

- ▶ `pre-commit` → Runs before a commit is created (linting, formatting)
- ▶ `pre-push` → Runs before pushing to a remote
- ▶ `commit-msg` → Validates commit message format

 **Pro Tip:** Tools like Husky make it easy to manage Git hooks in JavaScript projects.

 **Real-World Use:** A pre-commit hook running ESLint/Prettier stops badly formatted code from ever being committed in the first place.

? Q36. When would a project need Git LFS (Large File Storage)?

 **Answer:**

Git LFS replaces large files like videos or datasets with lightweight pointers in Git, while storing the actual content on a remote server.

 **Pro Tip:** Use Git LFS to avoid bloating repository size with large binary assets.

 **Real-World Use:** Game studios and ML teams use LFS to version large assets/datasets without bloating the core Git repository to gigabytes.

? Q37. What branch protection rules would you set up for a production repo?

 **Answer:**

Protected branches enforce rules such as requiring PR reviews, passing status checks, and restricting force pushes before merging.

 **Pro Tip:** Protect your main/production branch to prevent accidental direct pushes.

 **Real-World Use:** Standard practice: require at least one approval, passing CI checks, and block force-pushes to main.

? Q38. How does your team use GitHub Issues in the development cycle?

 **Answer:**

GitHub Issues track bugs, feature requests, and tasks. They can be labeled, assigned, and linked to pull requests.

 **Pro Tip:** Use keywords like 'Fixes #12' in a PR description to auto-close linked issues on merge.

 **Real-World Use:** Product managers file Issues for bugs/features, engineers self-assign them, and link the fixing PR to auto-close it.

? Q39. How would a GitHub Project board fit into agile sprint planning?

 **Answer:**

GitHub Projects are Kanban-style boards that organize issues and pull requests across stages of work.

 **Pro Tip:** Automate card movement using built-in workflows when issues are closed or PRs merged.

 **Real-World Use:** Teams mirror their sprint board (To Do → In Progress → Review → Done) directly in GitHub Projects instead of a separate tool.

GitHub & Real-World Workflows (Q40–Q52)

? Q40. Why would a large repo need a CODEOWNERS file?

 **Answer:**

CODEOWNERS defines individuals or teams responsible for specific files/directories, and auto-requests their review on relevant PRs.

 **Pro Tip:** Place it in the root, docs/, or .github/ directory for GitHub to recognize it.

 **Real-World Use:** In a big org, CODEOWNERS auto-assigns the right team (backend vs frontend) as reviewers so PRs don't go unreviewed.

? Q41. When would you use full Gitflow vs a simpler branching strategy?

💬 Answer:

Gitflow uses dedicated branches: main, develop, feature/*, release/*, and hotfix/* to organize the development lifecycle.

💡 **Pro Tip:** For simpler projects, many teams now prefer trunk-based development over full Gitflow.

🌐 **Real-World Use:** Gitflow suits apps with scheduled releases (e.g. mobile apps); fast-moving SaaS startups usually prefer simpler trunk-based development.

? Q42. When is git clean genuinely useful during development?

💬 Answer:

⚙️ **Commands / Steps:**

- ▶ git clean -n → Preview what would be deleted (dry run)
- ▶ git clean -f → Force delete untracked files
- ▶ git clean -fd → Also delete untracked directories
- ▶ git clean -fdx → Also remove gitignored files

💡 **Pro Tip:** Always run git clean -n first to avoid accidentally deleting important files.

🌐 **Real-World Use:** After switching branches repeatedly, leftover build files cause confusing errors — git clean -fd wipes the working directory to a known state.

? Q43. Why would you need git stash -u over a normal stash?

💬 Answer:

⚙️ **Commands / Steps:**

- ▶ git stash → Stashes tracked, modified files only.

- ▶ `git stash -u` → Also stashes untracked (new) files.
- ▶ `git stash save 'message'` → Label a stash for easy identification.

 **Pro Tip:** Use `git stash save 'WIP: login feature'` to label stashes for easy identification.

 **Real-World Use:** If you generated new config or log files mid-task and need a totally clean working directory to test something, `-u` stashes those too.

? Q44. When would git worktree save you time over constantly switching branches?

 **Answer:**

git worktree allows you to check out multiple branches of the same repository into separate working directories simultaneously.

 **Pro Tip:** Useful for testing a hotfix branch without disturbing your current feature branch.

 **Real-World Use:** Reviewing a teammate's PR while keeping your own unfinished feature checked out in another folder, without stashing anything.

? Q45. SSH vs HTTPS for GitHub remotes — which do you use and why?

 **Answer:**

 **Commands / Steps:**

- ▶ HTTPS → Uses username/token for auth. Easy to set up but requires token entry.
- ▶ SSH → Uses key pairs for auth. No need to enter credentials on every push/pull.

 **Pro Tip:** SSH is generally preferred for frequent push/pull workflows once keys are configured.

 **Real-World Use:** Most professional setups standardize on SSH keys per developer machine to avoid typing tokens on every push.

? Q46. How does your team handle versioned releases on GitHub?

💬 Answer:

A GitHub Release packages a specific tagged version of your code, with release notes and downloadable build artifacts.

💡 **Pro Tip:** Follow semantic versioning (e.g., v1.2.0 = major.minor.patch) for predictable release tags.

🌐 **Real-World Use:** On release day, you tag the commit, generate a GitHub Release with changelog notes, and trigger a deploy workflow off that tag.

? Q47. Have you deployed anything using GitHub Pages?

💬 Answer:

GitHub Pages is a free static site hosting service that publishes content directly from a GitHub repository.

💡 **Pro Tip:** Pages can build automatically from a docs/ folder or a dedicated gh-pages branch.

🌐 **Real-World Use:** Common for hosting project documentation, a portfolio site, or a frontend demo directly from the repo with zero extra hosting cost.

? Q48. How do you keep API keys safe inside a GitHub Actions workflow?

💬 Answer:

Secrets are encrypted environment variables stored at the repository or organization level, used to pass sensitive data into workflows.

💡 **Pro Tip:** Never hardcode credentials in workflow YAML files — always reference them via `${{ secrets.MY_KEY }}`.

 **Real-World Use:** Database passwords and API tokens go into encrypted Repo Secrets, and are injected into the workflow at runtime securely.

? Q49. When would you ask a teammate to squash their commits before merging?

 **Answer:**

A squash merge combines all commits from a feature branch into a single clean commit before merging, keeping history readable.

 **Pro Tip:** Squash merging is great for feature branches with many small 'WIP' or 'fix typo' commits.

 **Real-World Use:** If their branch has 15 'fix typo' and 'wip' commits, squash-merging keeps the main branch history readable for everyone else.

? Q50. How do you keep your fork updated with the original project?

 **Answer:**

 **Commands / Steps:**

- ▶ git remote add upstream <original-repo-url>
- ▶ git fetch upstream
- ▶ git checkout main
- ▶ git merge upstream/main
- ▶ git push origin main

 **Pro Tip:** Sync your fork regularly before opening a PR to avoid stale, conflict-heavy submissions.

 **Real-World Use:** Open-source contributors regularly sync from upstream to avoid conflict-heavy PRs from stale forks.

? Q51. You accidentally deleted a branch — how do you get it back?

💬 Answer:

git reflog records updates to the tip of branches and HEAD, even commits not reachable from any branch, letting you recover lost commits.

⚙️ Commands / Steps:

- ▶ git reflog → Find the lost commit hash
- ▶ git checkout -b <branch-name> <hash> → Restore it as a new branch

💡 **Pro Tip:** If you accidentally reset or delete a branch, git reflog can often save the day within a session.

🌐 **Real-World Use:** This is a classic 'panic' interview scenario — git reflog plus a quick checkout of the lost commit hash saves the day.

? Q52. Walk me through your complete open-source contribution workflow.

💬 Answer:

⚙️ Commands / Steps:

- ▶ Fork the original repository on GitHub
- ▶ Clone your fork locally: git clone <your-fork-url>
- ▶ Add upstream remote: git remote add upstream <original-url>
- ▶ Create a feature branch: git checkout -b fix/issue-42
- ▶ Make your changes, commit with clear messages
- ▶ Push to your fork: git push origin fix/issue-42
- ▶ Open a Pull Request from your fork to the original repo

💡 **Pro Tip:** Always check the project's CONTRIBUTING.md before submitting your first PR.

🌐 **Real-World Use:** Fork → clone fork → add upstream → branch → code → push → PR to original is the standard real-world open-source flow.